Running Parallel C++ and Python Programs

Ubuntu Linux inside VMware Workstation / Player

A practical, command-by-command guide: prepare Ubuntu, install compilers and Python tools, run C++ with OpenMP, and run Python with multiprocessing.

Assumptions: You have a working Ubuntu virtual machine and can log in with a user account that has sudo privileges. Commands are intended for Ubuntu 22.04/24.04 and similar recent releases.

Important: “Parallel programming” means work runs across multiple CPU cores/processes. In a VM, the number of usable CPU cores is limited by the VM’s processor setting and the host computer’s available resources.

Quick path

1. Check the VM and Ubuntu → 2. Install build tools → 3. Test C++ → 4. Run C++ OpenMP in parallel → 5. Run Python multiprocessing → 6. Verify CPU usage and troubleshoot.

1. Set up VMware and Ubuntu correctly

In VMware's virtual-machine settings, shut down the VM first, then open its processor settings. Assign 2 or more virtual CPU cores if your host computer can spare them. Do not assign every host core; leave resources for the host OS. Start Ubuntu again.

Inside Ubuntu, open Terminal with **Ctrl+Alt+T**. Check the OS and visible CPU count:

```
cat /etc/os-release
uname -a
nproc
lscpu
```

The output of nproc is the number of processing units visible to your Ubuntu VM. If it shows 1, increase the VM's CPU allocation in VMware settings (when supported by your host hardware).

Update Ubuntu and install tools

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y build-essential g++ gcc make python3 python3-pip python3-venv
```

Confirm versions:

```
g++ --version
gcc --version
python3 --version
make --version
```

If a command asks for your password, type your Ubuntu login password. The terminal normally does not show characters while you type; press Enter when finished.

2. Make a working folder

```
mkdir -p ~/parallel-lab
cd ~/parallel-lab
pwd
```

The final command should print a path similar to /home/your-user/parallel-lab. Save the code files in this folder.

3. Run a basic C++ program (serial test)

Create a file named hello.cpp:

```
cat > hello.cpp <<'EOF'
#include <iostream>
int main() {
    std::cout << "Hello from C++ on Ubuntu!\n";
    return 0;
}
EOF
```

Compile and run it:

```
g++ -std=c++17 -O2 -Wall -Wextra hello.cpp -o hello
./hello
```

Expected output: Hello from C++ on Ubuntu!. The compiler creates an executable called hello.

4. Run parallel C++ using OpenMP

OpenMP is a common way to parallelize C/C++ loops on shared-memory CPU cores. Save the following as parallel.cpp:

```
#include <iostream>
#include <omp.h>

int main() {
    #pragma omp parallel
```

```
    {
        int thread_id = omp_get_thread_num();
        int thread_count = omp_get_num_threads();

        #pragma omp critical
        {
            std::cout << "Hello from thread " << thread_id
                      << " of " << thread_count << "\n";
        }
    }
    return 0;
}
```

Compile with the OpenMP flag -fopenmp, then run:

```
g++ -std=c++17 -O2 -Wall -Wextra -fopenmp parallel.cpp -o parallel
./parallel
```

The program should print one line per OpenMP thread. The order of lines can change each run because threads execute concurrently.

Choose a thread count

```
OMP_NUM_THREADS=4 ./parallel
# Or let OpenMP choose a default:
./parallel
```

Use a number no greater than the CPU capacity you intend to use. The exact count is controlled by OpenMP settings and runtime defaults; a VM exposing only one CPU core will not provide true multi-core execution.

5. Run parallel Python using multiprocessing

Python's multiprocessing module runs work in separate processes, which is useful for CPU-heavy tasks. Create parallel.py:

```
from multiprocessing import Pool, cpu_count
import os

def square(number):
    return (number, number * number, os.getpid())

if __name__ == "__main__":
    worker_count = min(4, cpu_count())
    numbers = list(range(1, 9))

    print(f"CPUs visible to Python: {cpu_count()}")
    print(f"Worker processes: {worker_count}")

    with Pool(processes=worker_count) as pool:
        results = pool.map(square, numbers)

    for number, result, process_id in results:
        print(f"{number} squared = {result} (PID {process_id})")
```

Run it with Python 3:

```
python3 parallel.py
```

You should see the CPU count, worker count, and results. Process IDs (PIDs) may repeat because a worker can handle more than one task. For this tiny example, parallel execution may not be faster; process startup has overhead.

Optional: use a virtual environment for Python packages

```
cd ~/parallel-lab
python3 -m venv .venv
source .venv/bin/activate
python --version
# When finished:
deactivate
```

The example uses only Python's standard library, so no pip package is required.

6. Check CPU/process activity

```
nproc
top
# If installed, htop is a friendlier process viewer:
sudo apt install -y htop
htop
```

Run top or htop while a longer-running parallel workload is active. A short program may finish before you can observe CPU activity.

7. Common errors and fixes

g++: command not found
Install the compiler: sudo apt update && sudo apt install -y build-essential g++

fatal error: omp.h: No such file or directory / OpenMP symbols missing
Use g++ and include the flag -fopenmp both when compiling and linking: g++ -fopenmp parallel.cpp -o parallel

Permission denied when running ./parallel
For a file you compiled yourself, check that you are in the correct folder and run ./parallel. Recompile if needed; do not use sudo to run ordinary programs.

python: command not found
On Ubuntu, use python3 parallel.py. Install it with sudo apt install -y python3 if missing.

Only one CPU appears
Check nproc and lscpu in Ubuntu, then shut down the VM and increase its virtual CPU allocation in VMware settings if your host supports it.

The parallel program is not faster
Small jobs can be slower due to startup, synchronization, and scheduling overhead. Try a large CPU-bound workload and compare timings.

8. Useful commands summary

```
# Update and install tools
sudo apt update
sudo apt install -y build-essential g++ python3 python3-pip python3-venv

# Create and enter workspace
mkdir -p ~/parallel-lab && cd ~/parallel-lab

# Compile/run regular C++
g++ -std=c++17 -O2 -Wall -Wextra hello.cpp -o hello
./hello

# Compile/run OpenMP C++
g++ -std=c++17 -O2 -Wall -Wextra -fopenmp parallel.cpp -o parallel
OMP_NUM_THREADS=4 ./parallel

# Run Python multiprocessing
python3 parallel.py

# Check CPU allocation
nproc
lscpu
```

Notes on safety and correctness

• Run these commands in the Ubuntu terminal, not in the Windows/macOS host terminal.
• Commands containing sudo make system-level changes; read them before running.
• Do not type the leading $ sometimes shown in tutorials; it is a prompt symbol, not part of the command.
• This guide uses CPU parallelism. GPU programming (CUDA/OpenCL), distributed computing (MPI), and Python threading are different topics with different setup requirements.

End of guide